

UNICESUMAR

Curso de Engenharia de Software

Disciplina: Programação Front End

Health Sync

Landing Page — Prontuário Médico Inteligente

Documentação do Projeto

Grupo:

João Paulo Campiolo Almeida — RA: 25001140-2

Matheus José de Azevedo — RA: 25102632-2

Joshua Filipe Rodrigues Guirado — RA: 25153351-2

Felipe Falaschi Cadedo — RA: 25036766-2

Douglas Meller Guirado — RA: 25068978-2

Guilherme Henrique Beitum Barbosa — RA: 25009441-2

Prof. Dacio F. Machado | 2026

1. Sobre o Projeto

O Health Sync surgiu de uma necessidade identificada pelo grupo durante a disciplina de Mentalidade Criativa e Empreendedora: a dificuldade que as pessoas têm de acessar e organizar seus próprios documentos de saúde. Exames perdidos, histórico médico espalhado em papel, carteirinha de vacinas desatualizada — são problemas que praticamente todo mundo já enfrentou.

A proposta é simples: uma plataforma onde o paciente tem tudo em um só lugar, com Inteligência Artificial integrada para ajudar a interpretar exames e responder dúvidas sobre saúde.

A entrega deste trabalho é a landing page do produto — uma página web pública que apresenta o Health Sync, suas funcionalidades e proposta de valor, desenvolvida com HTML5, CSS3 e JavaScript puro.

1.1 Acesso ao Projeto

- Site publicado: <https://flourishing-mousse-5947e1.netlify.app/>
- Repositório:

1.2 Equipe

Nome	RA
João Paulo Campiolo Almeida	25001140-2
Matheus José de Azevedo	25102632-2
Joshua Filipe Rodrigues Guirado	25153351-2
Felipe Falaschi Cadedo	25036766-2
Douglas Meller Guirado	25068978-2
Guilherme Henrique Beitum Barbosa	25009441-2

2. Tecnologias Utilizadas

A decisão de usar HTML, CSS e JavaScript puro — sem frameworks — foi intencional. O objetivo era ter controle total sobre o código, entender cada linha do que foi escrito, e produzir algo limpo e fácil de manter. Frameworks como React ou Angular trazem complexidade desnecessária para uma landing page estática.

Tecnologia	Versão / Tipo	Função no Projeto
HTML5	Linguagem de marcação	Estrutura semântica da página
CSS3	Folha de estilos	Visual, animações e responsividade

JavaScript (ES6+)	Linguagem de script	Interatividade e comportamento dinâmico
Phosphor Icons	Biblioteca de ícones (CDN)	Ícones SVG usados na interface
Netlify	Hospedagem estática	Deploy e disponibilização pública da página
GitHub	Controle de versão	Repositório do código-fonte

2.1 Por que HTML, CSS e JS puro?

Usar tecnologias sem dependências externas tem vantagens claras: carregamento mais rápido, código mais simples de inspecionar, e zero necessidade de build ou configuração de ambiente. Para uma landing page cujo objetivo é apresentar um produto, essa é a escolha mais coerente.

Os ícones são carregados via CDN da biblioteca Phosphor Icons, o que elimina a necessidade de instalar qualquer pacote localmente.

3. Organização do Projeto

O projeto segue uma estrutura de pastas simples e direta, com separação clara entre HTML, CSS e JavaScript:

```
HealthSync_LandingPage/
├── index.html           → estrutura e conteúdo da
│                       │   página
├── css/                → todo o visual, animações e
│   └── style.css       →
├── js/                 → comportamento e
│   └── main.js         →
└──
```

Cada arquivo tem uma responsabilidade única e bem definida: o HTML cuida da estrutura, o CSS cuida do visual e o JavaScript cuida do comportamento. Essa separação é uma das práticas fundamentais do desenvolvimento front-end.

4. Seções da Página

A landing page foi dividida em quatro seções principais, cada uma com um objetivo específico dentro da jornada do visitante:

Seção	Tag HTML	Objetivo
Navbar	<header class="navbar">	Navegação fixa no topo com links de âncora e menu mobile
Hero	<section class="hero">	Primeiro impacto — slogan, proposta de valor e mockup do chat com IA
Funcionalidades	<section class="features">	Cards descritivos das 4 funcionalidades principais do produto
Como Funciona	<section class="how-it-works">	Passo a passo visual do fluxo do usuário

Rodapé	<footer class="footer">	Links, redes sociais e créditos do projeto
--------	-------------------------	--

Todas as seções usam tags semânticas do HTML5 — header, section, footer, nav — em vez de divs genéricas. Isso melhora o SEO da página e facilita a leitura do código por outros desenvolvedores.

5. Decisões de Layout e Interface

5.1 Identidade Visual

A página adota um tema escuro (dark mode) com fundo preto (#09090b), que transmite modernidade e contrasta bem com os elementos em azul, roxo e ciano. As três cores principais formam o gradiente de marca do produto:

- Azul (#3b82f6) — cor primária, representa tecnologia e confiança
- Roxo (#8b5cf6) — cor secundária, traz sofisticação
- Ciano (#06b6d4) — cor de destaque, usada em elementos de ação

Esse gradiente é definido uma única vez no arquivo CSS usando uma variável (--gradient-brand) e reutilizado em botões, títulos e elementos de destaque ao longo de toda a página.

5.2 Glassmorphism

Os cards e o mockup da Hero Section usam o estilo glassmorphism — fundo semitransparente com efeito de vidro fosco (backdrop-filter: blur). Essa escolha visual reforça o conceito de tecnologia avançada que o produto quer transmitir.

O efeito é implementado com três propriedades CSS em conjunto: background com opacidade baixa, backdrop-filter: blur() para o desfoque, e border com transparência para simular a borda do vidro.

5.3 Tipografia

O projeto usa duas famílias tipográficas definidas em variáveis CSS: Segoe UI (e similares) para títulos e headings, e Arial (e similares) para texto corrido e navegação. Ambas são fontes seguras que funcionam em todos os sistemas sem necessidade de carregamento externo.

5.4 Responsividade

A página foi desenvolvida para funcionar em qualquer dispositivo. Isso foi feito com duas abordagens complementares:

- CSS Grid com repeat(auto-fit, minmax()) na seção de funcionalidades — os cards se reorganizam automaticamente conforme o espaço disponível
- Media queries nos breakpoints de 992px (tablet) e 768px (mobile) — ajustam o layout de duas colunas para uma coluna, ocultam o menu desktop e exibem o botão hamburger

No mobile, o menu de navegação é substituído por um painel lateral que desliza da lateral direita, controlado pelo JavaScript.

6. JavaScript — Funcionalidades Implementadas

O arquivo main.js tem quatro funcionalidades bem separadas, cada uma com seu bloco de código e comentário explicativo:

Nº	Funcionalidade	Como funciona
1	Efeito de scroll na navbar	Escuta o evento scroll da window. Quando o usuário rola mais de 50px, adiciona a classe <code>.scrolled</code> ao header, que aplica fundo semitransparente e blur via CSS
2	Menu hamburguer mobile	Três event listeners: um para abrir o menu (adiciona <code>.active</code>), um para fechar pelo botão X, e um loop em todos os links para fechar ao clicar em qualquer item do menu
3	Animações ao rolar (Intersection Observer)	Observa todos os elementos com classes <code>.reveal-up</code> , <code>.reveal-left</code> e <code>.reveal-right</code> . Quando o elemento entra na tela, adiciona a classe <code>.active</code> que aciona a transição CSS de opacity e transform
4	Smooth scroll	Intercepta o clique em todos os links com href começando em <code>#</code> , calcula a posição do elemento destino com <code>getBoundingClientRect()</code> e usa <code>window.scrollTo()</code> com <code>behavior: smooth</code>

Todo o código JavaScript é executado dentro de um `DOMContentLoaded`, garantindo que o script só roda depois que a página inteira foi carregada. Isso evita erros de referência a elementos que ainda não existem no DOM.

7. Boas Práticas Aplicadas no CSS

7.1 Variáveis CSS

Todas as cores, fontes e transições do projeto são definidas como variáveis no bloco `:root` no início do arquivo `style.css`. Isso centraliza as decisões visuais em um único lugar — se precisar mudar a cor primária, altera em um ponto e afeta toda a página.

7.2 Animações de Entrada

Os elementos da página aparecem com animação quando entram na área visível do usuário. As classes `.reveal-up`, `.reveal-left` e `.reveal-right` definem o estado inicial (invisible + deslocado). Quando o JavaScript adiciona a classe `.active`, o CSS faz a transição suave para o estado final (visível + posição correta) usando `transition` com `cubic-bezier`.

7.3 Animação Flutuante

Os elementos flutuantes na seção 'Como Funciona' usam um `@keyframes` float que oscila o elemento 15px para cima e para baixo em loop de 6 segundos. O segundo elemento tem `animation-delay: -3s` para criar a sensação de que os dois elementos flutuam de forma independente.

7.4 Organização do Arquivo

O `style.css` tem 760 linhas organizadas em blocos separados por comentários. Cada bloco cuida de uma parte específica: reset global, tipografia, botões, navbar, hero, features, how it works, footer, animações e media queries. Essa organização facilita a localização de qualquer trecho do código.

8. Hospedagem

A página está hospedada na Netlify, plataforma de hospedagem estática com integração direta ao GitHub. A escolha se deu pelos seguintes motivos:

- Deploy automático: qualquer push no repositório publica a nova versão automaticamente
- HTTPS gratuito: certificado SSL ativo, conexão segura sem configuração manual
- CDN global: os arquivos são distribuídos em servidores ao redor do mundo, garantindo velocidade de carregamento independente da localização do usuário
- Plano gratuito adequado: sem limitações para projetos estáticos como este

O endereço público da página é: <https://flourishing-mousse-5947e1.netlify.app/>

9. Requisitos do Sistema

9.1 Requisitos Funcionais

ID	Requisito	Status
RF01	Exibir navbar fixa com links de navegação por âncora	Implementado
RF02	Menu hamburguer funcional em dispositivos móveis	Implementado
RF03	Hero Section com slogan, subtítulo e mockup do chat com IA	Implementado
RF04	Seção de funcionalidades em grid de cards	Implementado
RF05	Seção 'Como Funciona' com passo a passo numerado	Implementado
RF06	Elementos com animação de entrada ao rolar a página	Implementado
RF07	Navegação com smooth scroll entre seções	Implementado

RF08	Rodapé com logo, descrição e links de redes sociais	Implementado
------	---	--------------

9.2 Requisitos Não Funcionais

ID	Requisito	Como foi Atendido
RNF01	Responsividade em mobile, tablet e desktop	Media queries nos breakpoints de 768px e 992px
RNF02	Carregamento rápido	Sem frameworks — apenas HTML, CSS e JS puro
RNF03	URL pública acessível	Netlify — https://flourishing-mousse-5947e1.netlify.app/
RNF04	Código-fonte disponível publicamente	GitHub — /JoshuaGuirado/HealthSync
RNF05	Conexão segura HTTPS	SSL automático da Netlify
RNF06	Compatibilidade com navegadores modernos	CSS e JS sem recursos experimentais

10. Conclusão

O desenvolvimento da landing page do Health Sync colocou em prática os principais conceitos estudados na disciplina de Programação Front End: estruturação semântica com HTML5, design visual e responsivo com CSS3, e interatividade com JavaScript.

A decisão de não usar frameworks foi acertada para o escopo do projeto — o resultado é uma página rápida, com código legível e totalmente compreendido por todos os membros do grupo. Cada funcionalidade implementada tem uma justificativa técnica clara e pode ser explicada linha a linha.

O projeto cumpre todos os requisitos propostos: página funcional e hospedada, código-fonte em repositório público, tecnologias aplicadas de forma padronizada e documentação descrevendo as decisões tomadas.

11. Referências

- MDN Web Docs — <https://developer.mozilla.org>
- Phosphor Icons — <https://phosphoricons.com>
- Netlify Docs — <https://docs.netlify.com>
- W3C — HTML5 Semantic Elements — https://www.w3schools.com/html/html5_semantic_elements.asp
- CSS Tricks — A Complete Guide to Grid — <https://css-tricks.com/snippets/css/complete-guide-grid/>

- MDN — Intersection Observer API — https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API